



Chapter 13

Sets and Union-Find

13.1 Sets and Set Operations

※ 13.1.1 Set Terminology

A **set** is a well-defined collection of *distinct* elements. In practice, sets are often used to check if a specific value exists within a group of elements. However, before we begin discussing use cases of sets, let us first introduce some set terminology:

- The **union** ($\cup$) of two sets consists of all elements that are in one set or the other. For example, if set A consists of the elements {1, 2, 3} and set B consists of the elements {1, 3, 5}, the union of sets A and B (denoted as $A \cup B$) is the set consisting of the elements {1, 2, 3, 5}.
- The **intersection** ($\cap$) of two sets consists of all elements that are members of both sets. Using the same example as above, the intersection of sets A and B (denoted as $A \cap B$) consists of the elements {1, 3}, since only 1 and 3 are members of both sets A and B.
- The **set difference** ($\setminus$ or $-$) of two sets consists of all elements that are members of one set but not the other. In the provided example, the set difference $A \setminus B$ (or $A - B$) consists of the element {2}, since 2 is the only element in set A that is not in set B. The set difference $B \setminus A$ (or $B - A$) consists of the element {5}, since 5 is the only element in set B that is not in set A.
- The **symmetric difference** ($\oplus$) of two sets consists of all elements that are in either set but not both. In the provided example, the symmetric difference $A \oplus B$ consists of the elements {2, 5}, since 2 and 5 are the only elements that exist in only one of A or B.

※ 13.1.2 Set Implementation

How would you implement a set? There are several ways to implement a set that efficiently supports the operations specified above. For instance, the STL implements its sets using hash tables (`std::unordered_set<>`) and self-balancing trees (`std::set<>`), both of which will be covered in later chapters. In this chapter, we will consider a simple set implementation that uses a sorted vector as its underlying container. Sorted vectors work well because binary search (see chapter 15) can be used to check an element's membership in $\Theta(\log(n))$ time.

To find the union of two sets whose elements are stored in a sorted vector, you would first initialize iterators to the beginning of both vectors. Then, compare the values at each of these iterators and push the smaller element to a destination vector representing the set union. The iterator pointing to the element that was pushed in should then be incremented. If the elements at both iterators are identical, only one copy of the element is pushed to the destination vector and both iterators are incremented (this is done to avoid duplication, which is not allowed in a set). This process is repeated until both iterators point to the end of their respective containers. If one iterator reaches the end before the other, iterate through the remaining elements of the other vector and push them into the destination vector.

To illustrate this, consider the following example. Here, set A contains the elements {3, 4, 6, 8, 9, 10} and set B contains the elements {2, 4, 5, 8}. Both sets are implemented using a sorted vector. We want to build the set $A \cup B$, which stores the elements in the union of A and B.

Set A	Set B	$A \cup B$
3	2	
4	4	
6	5	
8	8	
9		
10		

First, we initialize two iterators at the beginning of the two underlying vectors. The two elements at these iterators are then compared, with the smaller value being sent to the output set $A \cup B$. This process continues until one of the iterators reaches the end of its respective sorted vector.

Set A	Set B	$A \cup B$
→ 3	→ 2	
4	4	
6	5	
8	8	
9		
10		

2 is smaller than 3, so we push 2 into the output vector and increment the iterator for set B.

Set A	Set B	$A \cup B$
→ 3	→ 4	2
4	5	
6	8	
8		
9		
10		

3 is smaller than 4, so we push 3 into the output vector and increment the iterator for set A.

Set A	Set B	$A \cup B$
→ 4	→ 4	2 3
6	5	
8	8	
9		
10		

Both iterators now point to the same value, so we push 4 into the output vector and increment both iterators.

Set A	Set B	$A \cup B$
→ 6	→ 5	2 3 4
8	8	
9		
10		

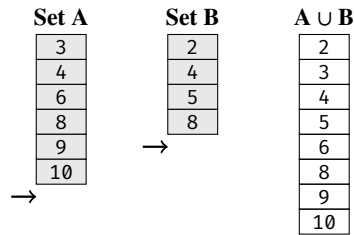
Continuing this process, we push 5 into the union, followed by 6. After these values are added, both iterators would point to 8.

Set A	Set B	$A \cup B$
→ 8	→ 8	2 3 4 5 6
9		
10		

Since both iterators point to the same value, we push 8 into the output vector and increment both iterators.

Set A	Set B	$A \cup B$
→ 9	→	2 3 4 5 6 8
10		

After 8 is added to the union, notice that the iterator for set B is pointing off the end of the underlying vector. When this happens, all the remaining elements in the other set (in this case, set A) are pushed into the union. This is okay since both sets are traversed in sorted order. If one set reaches the end before another, the other set must contain elements that are larger than the contents of the set that reached the end. Thus, the output vector would remain sorted and all values would remain unique.



A potential implementation of the set union process is shown below. The following code takes in iterators to the underlying containers of two sets and an iterator pointing to the output vector. The comparator `comp(a, b)` returns true if `a` is less than `b`.

```

1  template <class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
2  OutputIterator set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2,
3                          OutputIterator result, Compare comp) {
4      // loop until one set reaches the end
5      while (first1 != last1 && first2 != last2) {
6          // if element at first1 is less than element at first2, add element at first1 to result and increment first1
7          if (comp(*first1, *first2)) {
8              *result++ = *first1++;
9          } // if
10         // else if element at first2 is less than element at first1, add element at first2 to result and increment first2
11         else if (comp(*first2, *first1)) {
12             *result++ = *first2++;
13         } // else if
14         // else if both elements are equal, add one copy of the element to result and increment both iterators
15         else {
16             *result++ = *first1++;
17             ++first2;
18         } // else
19     } // while
20     // if first2 reaches last2 first, copy remaining elements in container 1 to the result set
21     while (first1 != last1) {
22         *result++ = *first1++;
23     } // while
24     // if first1 reaches last1 first, copy remaining elements in container 2 to the result set
25     while (first2 != last2) {
26         *result++ = *first2++;
27     } // while
28     // returns iterator pointing one past the end of the sorted union (expected for standard algorithm functions)
29     return result;
30 } // set_union()

```

What is the time complexity of the `set_union()` function? With each iteration, either `first1` or `first2` is incremented. As a result, if m and n are the sizes of the two sets, the algorithm iterates through a total of $m + n$ elements. Thus, `set_union()` runs in $\Theta(m + n)$ time.

How would we change the `set_union()` algorithm to a `set_intersection()` algorithm? When dealing with a set intersection, we only add the element to the output vector if it belongs in both sets. Thus, only the `else` block on line 15 of the previous code should write the element to the output vector. We can also get rid of lines 20 to 27; once the first set reaches the end, we know that the elements remaining in the other set cannot be in the set intersection. This leads us to the following modified code for `set_intersection()`.

```

1  template <class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
2  OutputIterator set_intersection(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
3                                 InputIterator2 last2, OutputIterator result, Compare comp) {
4      // loop until one set reaches the end
5      while (first1 != last1 && first2 != last2) {
6          // if element at first1 is less than element at first2, do not add to result and only increment first1
7          if (comp(*first1, *first2)) {
8              ++first1;
9          } // if
10         // else if element at first2 is less than element at first1, do not add to result and only increment first2
11         else if (comp(*first2, *first1)) {
12             ++first2;
13         } // else if
14         // else if both elements are equal, add one copy of the element to result and increment both iterators
15         else {
16             *result++ = *first1++;
17             ++first2;
18         } // else
19     } // while
20     // return iterator that points one past the end of the sorted union
21     return result;
22 } // set_intersection()

```

The same template can be used to implement `set_difference()` and `set_symmetric_difference()`. The only difference is in the values that are added to the result vector. In the case of `set_difference()`, only the if block on line 7 of the `set_union()` code should write its value to the result vector, and the while loop on lines 21-23 should be kept.

```

1  template <class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
2  OutputIterator set_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
3                               InputIterator2 last2, OutputIterator result, Compare comp) {
4      // loop until one set reaches the end
5      while (first1 != last1 && first2 != last2) {
6          // if element at first1 is less than element at first2, add element at first1 to result and increment first1
7          if (comp(*first1, *first2)) {
8              *result++ = *first1++;
9          } // if
10         // else if element at first2 is less than element at first1, increment first2
11         else if (comp(*first2, *first1)) {
12             ++first2;
13         } // else if
14         // else if both elements are equal, increment both iterators
15         else {
16             ++first1;
17             ++first2;
18         } // else
19     } // while
20     // if first2 reaches last2 first, copy remaining elts in ctn 1 to result set
21     while (first1 != last1) {
22         *result++ = *first1++;
23     } // while
24     // returns iterator that points one past the end of the sorted union
25     return result;
26 } // set_difference()

```

The implementation of `set_symmetric_difference()` (elements that exist in either set but not both) is also similar to the implementation of `set_union()`, except that the else block on line 15 should *not* write a value to the result vector. The code is shown below:

```

1  template <class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
2  OutputIterator set_symmetric_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
3                                          InputIterator2 last2, OutputIterator result, Compare comp) {
4      while (first1 != last1 && first2 != last2) {
5          if (comp(*first1, *first2)) {
6              *result++ = *first1++;
7          } // if
8          else if (comp(*first2, *first1)) {
9              *result++ = *first2++;
10         } // else if
11         else {
12             ++first1;
13             ++first2;
14         } // else
15     } // while
16     while (first1 != last1) {
17         *result++ = *first1++;
18     } // while
19     while (first2 != last2) {
20         *result++ = *first2++;
21     } // while
22     return result;
23 } // set_symmetric_difference()

```

※ 13.1.3 Set Operations in the STL

These four operations for sorted sets: `set_union()`, `set_intersection()`, `set_difference()`, and `set_symmetric_difference()`, are all implemented in the STL's `<algorithm>` library and behave similarly to the examples above. All four functions take in iterator ranges to two *sorted* sequence containers, an output iterator that points to the destination container, and an optional comparator. These four functions all return an iterator one past the last element of the constructed range.

```

template <typename InputIterator1, typename InputIterator2, typename OutputIterator, typename Compare>
OutputIterator std::set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2,
                             OutputIterator result, Compare comp);

```

Constructs a sorted range beginning at `result` with the contents of the set union of the elements in the two sorted ranges `[first1, last1)` and `[first2, last2)` and returns an iterator one past the last element in the constructed range. The comparator is optional, and `operator<` is used by default if it is not specified.

```

template <typename InputIterator1, typename InputIterator2, typename OutputIterator, typename Compare>
OutputIterator std::set_intersection(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
                                     InputIterator2 last2, OutputIterator result, Compare comp);

```

Constructs a sorted range beginning at `result` with the contents of the set intersection of the elements in the two sorted ranges `[first1, last1)` and `[first2, last2)` and returns an iterator one past the last element in the constructed range. The comparator is optional, and `operator<` is used if it is not specified.

```
template <typename InputIterator1, typename InputIterator2, typename OutputIterator, typename Compare>
OutputIterator std::set_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
                                InputIterator2 last2, OutputIterator result, Compare comp);
```

Constructs a sorted range beginning at `result` with the contents of the set difference of the elements in the sorted range `[first1, last1)` with respect to the elements in the sorted range `[first2, last2)` and returns an iterator one past the last element in the constructed range. The comparator is optional, and `operator<` is used by default if it is not specified.

```
template <typename InputIterator1, typename InputIterator2, typename OutputIterator, typename Compare>
OutputIterator std::set_symmetric_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2,
                                           InputIterator2 last2, OutputIterator result, Compare comp);
```

Constructs a sorted range beginning at `result` with the contents of the set symmetric difference of the elements in the two sorted ranges `[first1, last1)` and `[first2, last2)` and returns an iterator one past the last element in the constructed range. The comparator is optional, and `operator<` is used if it is not specified.

To demonstrate an example, the following code uses `std::set_union()` in the `<algorithm>` library.

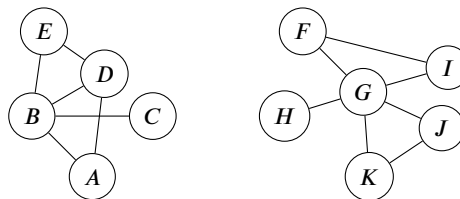
```
1  std::vector<int32_t> v1 = {1, 2, 3};
2  std::vector<int32_t> v2 = {1, 3, 5};
3  std::vector<int32_t> v_union;
4  std::set_union(v1.begin(), v1.end(), v2.begin(), v2.end(), std::back_inserter(v_union));
5  for (int32_t val : v_union) {
6      std::cout << val << " "; // prints 1 2 3 5
7  } // for val
```

13.2 Union-Find and Path Compression

※ 13.2.1 Disjoint Sets and the Union-Find Data Structure

In the previous section, we introduced the concept of a set. One benefit of using a set is that we are able to quickly determine if any element is part of that set. However, certain problems go beyond simply querying if an element exists in a set.

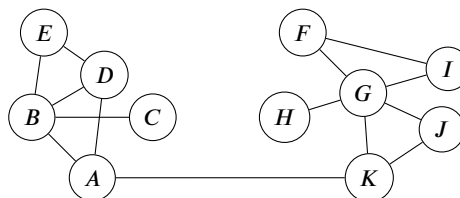
One such problem involves something known as a **disjoint set**. We consider two sets to be disjoint if they do not share any elements. In disjoint set problems, we are often required to organize a collection of objects into different groups and efficiently determine whether two objects belong to the same group. As an example, suppose we wanted to implement a program that can determine whether two computers in a network are connected to each other. Consider the following diagram, where each node represents a computer, and each edge represents a network connection that exists between two computers. Under this setup, we have two disjoint sets: one that includes computers *A-E*, and one that includes computers *F-K*.



Suppose we were asked if data can be sent through the network from computer *C* to computer *E*. How would we answer this question? In this case, the answer would be yes, since computers *C* and *E* belong to the same disjoint set; to send data from *C* to *E*, we could first send the data from *C* to *B*, and then from *B* to *E*.

What if we were instead asked if data could be sent through the network from computer *C* to computer *I*? The answer to this question would be no, since computers *C* and *I* belong to two different disjoint sets — there is no way to use the existing connections to travel from *C* to *I*.

Now, let's suppose we added a connection between computers *A* and *K*, as shown below.



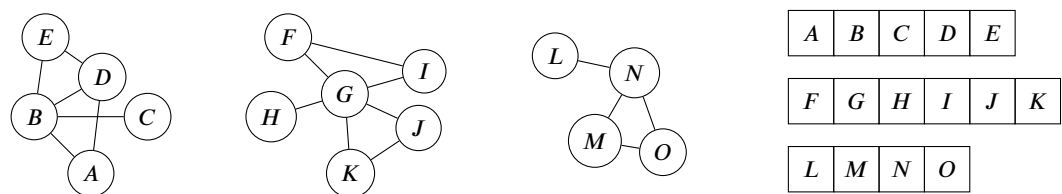
What happens to our answers to the previous two questions? In the case of computers *C* and *E*, the answer would still be yes, since *C* and *E* still belong to the same disjoint set. However, the answer for *C* and *I* would change to yes, since the new connection ended up joining *C* and *I* into the same disjoint set! Even though neither *C* nor *E* had their connections directly modified, an external connection caused their disjoint set membership to be the same.

If we want to efficiently solve problems involving disjoint sets, we would need to store our data in a container that allows us to keep track of multiple disjoint sets and membership within these sets. This is done using a container known as a **union-find data structure** (alternatively known as a *disjoint-set data structure*). The union-find data structure is a container that relies on two primary operations, union and find:

- `union_set(x, y)` is used to merge the set containing element *x* with the set containing element *y*. Using the above example, `union_set(A, K)` would add a connection between computers *A* and *K*.¹
- `find_set(x)` is used to identify the disjoint set that element *x* belongs to.

¹The reason we are using `union_set()` and not `union()` is that the keyword `union` is reserved in C++ and cannot be used to name a custom function or variable. Similarly, `set_union()` is not used because that is the name of a function in the STL algorithm library.

How should we implement a union-find data structure to support efficient union and find operations? The simplest approach would be to use the same sorted vector approach to keep track of our disjoint sets. Each disjoint set would be stored in its own sorted vector, as shown:

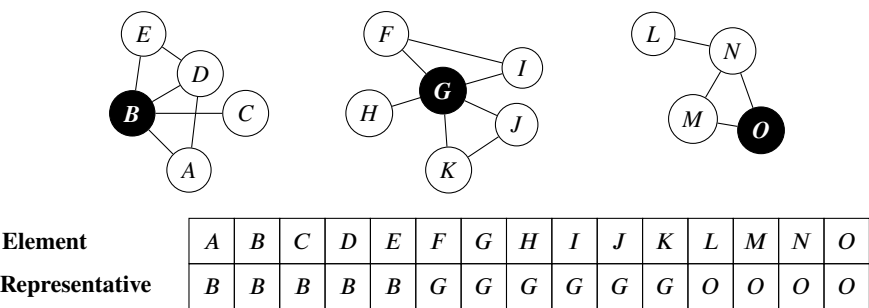


How would `union_set()` and `find_set()` be implemented using this approach? If `union_set()` is called on two elements, we would first check if the two elements are in the same sorted vector. If they are not, we would have to merge all the elements in one vector into the other. For example, if we wanted to union computers *A* and *K*, we would have to merge the first two sorted vectors (*A*-*E* and *F*-*K*). This takes linear time. If `find_set()` is called on an element, we would have to check every vector for the existence of that element. In the worst case, this also takes linear time. Because both union and find are linear-time operations, this implementation is inefficient, especially as the number of disjoint sets increases. It would be better to store all the elements in the *same* container, rather than a separate container for each disjoint set.

However, if we do store all our data in a single container, we will need to store additional information to keep track of which disjoint set each element belongs to. We can do this using a *representative system*, where each element in the union-find data structure remembers a unique representative that identifies which set it belongs in.

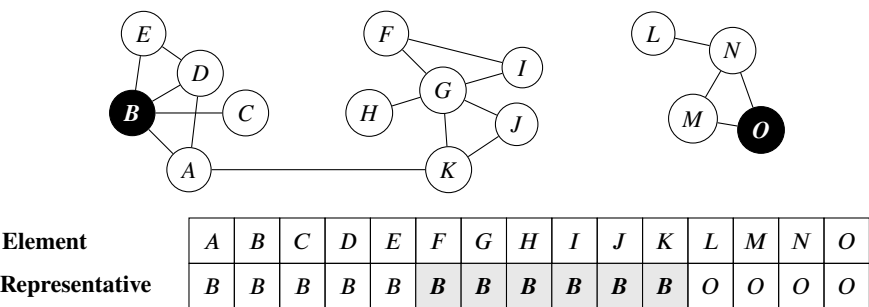
To use an analogy, suppose you are in a room full of people from all over the world, and you want to organize the people into disjoint sets based on citizenship. If everyone is mingled together, how would you be able to identify which country everyone is from? One way to solve this problem is to ask each person who the leader of their country is. If they answer with the name of the president of the United States, they must be from the United States. If they answer with the name of the Canadian Prime Minister, they must be from Canada. You can then use this information to organize the people into their correct groups.

The representative system works similarly for data in a union-find data structure. Each element in a disjoint set is assigned a representative that acts as the "leader" of its set. Every member of the set then remembers who their representative is. An example is shown below, where computers *B*, *G*, and *O* are arbitrarily chosen as the representative of each of the disjoint sets.



Here, computer *B* serves as the representative of all computers in the first disjoint set, computer *G* serves as the representative of all computers in the second disjoint set, and computer *O* serves as the representative of all computers in the third disjoint set. Compared to the previous implementation that kept a sorted vector for each disjoint set, the `find_set()` operation is much more efficient with this approach. To determine which set an element belongs to, `find_set()` would just need to check that element's representative. To determine if two elements are in the same disjoint set, simply compare their representatives. This procedure of forcing each element to remember its ultimate representative is known as the *quick-find* implementation of union-find, since `find_set()` can be completed in $\Theta(1)$ time.

However, the union operation is still inefficient here. If we wanted to union computers *A* and *K* together, we would have to change the representatives of all the elements in one of the sets — this would require a linear traversal of the container.



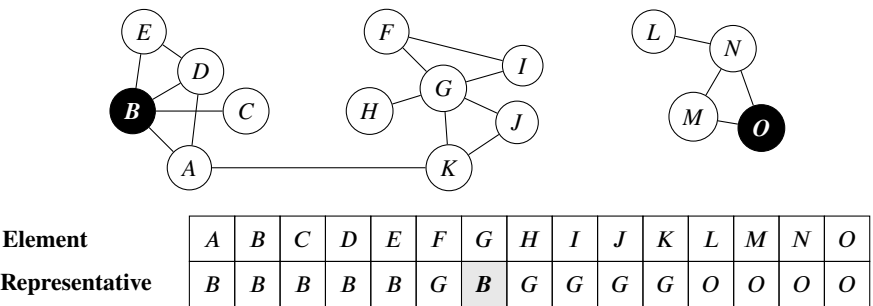
We can make the union process faster by removing the need for each element to remember its ultimate representative. It is okay if each element remembers an element that may *not* be the ultimate representative itself, as long as the element it remembers either (1) *directly* knows who the ultimate representative is or (2) *indirectly* knows another element that knows who the ultimate representative is. The ultimate representative serves as the leader of the entire disjoint set, and its representative is itself. This is known as the *quick-union* implementation of union-find.

Returning to the citizenship analogy, this process is similar to asking everyone to name a person who is a citizen of the same country rather than a direct leader (or ultimate representative). The idea here is that we will eventually be able to figure out which country each person belongs to

just by following the chain of representatives (e.g., if Alice says Bob is her representative, Bob says Cathy is his representative, and Cathy says that the president of the United States is her representative, we would know that Alice, Bob, and Cathy are all citizens of the United States).

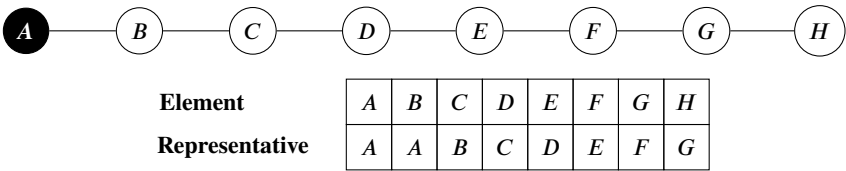
When using this implementation, a call to `union_set()` would only need to modify the ultimate representative of one of the disjoint sets that was merged. In the example, the ultimate representative of the first disjoint set was computer *B*, and the ultimate representative of the second disjoint set was computer *G*. If a connection were added between computers *A* and *K*, we would first check if *A* and *K* are already part the of the same disjoint set by comparing their ultimate representatives using two calls to `find_set()`. If they are not part of the same set, we would modify the ultimate representative of either *A* or *K* so that it refers to the ultimate representative of the other disjoint set.

In the following example, since *K*'s ultimate representative is *G* and *A*'s ultimate representative is *B*, we could change *G*'s representative to *B*. Note that we could have changed *B*'s representative to *G* instead; the decision to change *G*'s representative instead of *B*'s was made arbitrarily.



This reduces the amount of work needed for union, since we no longer need to traverse the entire container and reset the representative of every element whose representative was *G*. However, the find operation will likely be slower than before, since we may need to traverse an entire chain of elements to find the disjoint set that an element belongs to. For instance, suppose we called `find` on computer *F*. We know that *F*'s representative is *G*. However, *G*'s representative was changed to *B*. Because of this, we know that computer *F* is a member of the disjoint set whose ultimate representative is *B* (we know *B* is an ultimate representative because its representative is itself).

However, this is a simple case. Since we removed the requirement for each element to directly remember its ultimate representative, we could end up with an orientation of nodes like this:

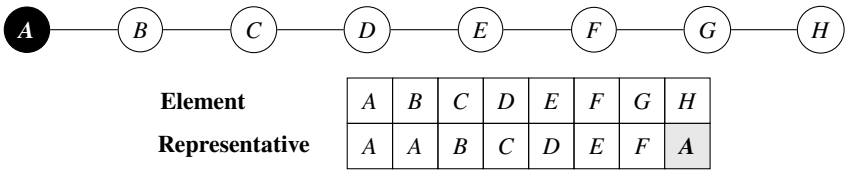


Suppose we called `find_set()` on computer *H* using this setup. *H*'s representative is *G*, but *G*'s representative is *F*, and *F*'s representative is *E*, ... we would have to traverse through every element in the container to figure out that *H*'s ultimate representative is computer *A*. This is a worst-case $\Theta(n)$ traversal for a single find operation! In addition, since union itself calls `find` twice (to determine the ultimate representatives of the two elements to join), the union operation here also runs in $\Theta(n)$ time (even if it is slightly faster on average in most situations). How can we adjust our implementation so that both union and find can be done efficiently? The solution is to use a strategy known as *path compression*.

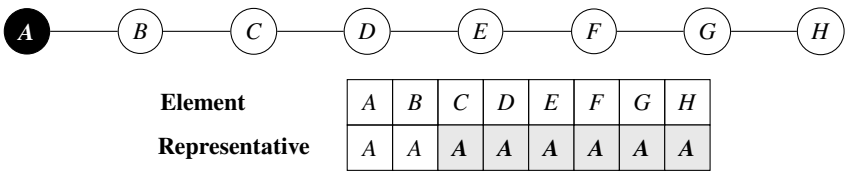
※ 13.2.2 Path Compression

When we forced each element to directly remember its ultimate representative, `find_set()` could be done quickly, but `union_set()` was inefficient. However, if we removed this requirement and allowed elements to store any representative that belongs to its own disjoint set, `union_set()` became faster but `find_set()` became less efficient.

A key insight to notice here is that, although it took a $\Theta(n)$ traversal to learn that computer *H*'s ultimate representative was computer *A*, this work only needs to be done once. If we call `find_set()` on *H* again, we should not need to traverse the entire chain of elements again; we already know that its ultimate representative is *A*! As a result, after calling `find_set()` on *H* the first time, we can speed up subsequent calls to `find_set()` by changing *H*'s representative to *A*, so that further `find_set()` calls on *H* would no longer require a $\Theta(n)$ traversal.

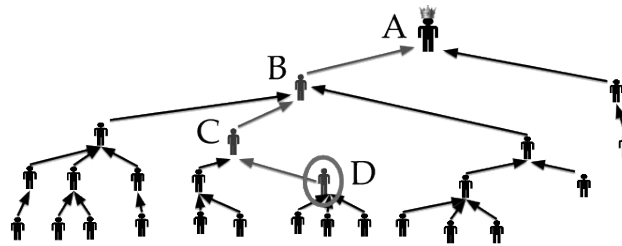


Yet, that is not all we can do! By traversing the entire chain, we learned more than the fact that *H*'s ultimate representative is *A*. Since *H*'s old representative was *G*, *H* and *G* must be part of the same disjoint set. Thus, if *H*'s ultimate representative is *A*, *G*'s ultimate representation must also be *A*. And, since *G*'s representative is *F*, *G* and *F* must be part of the same disjoint set, so *F*'s ultimate representative must also be *A*. Not only can we change the representative of *H* to *A*, but we can change the representative of *every element on the entire chain* to *A* as well!

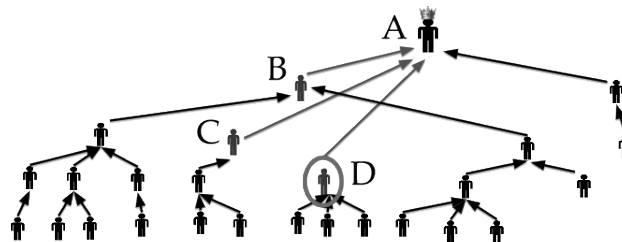


Now, every subsequent call to `find_set()` can be done quicker since every node now stores its ultimate representative. This process of moving elements closer to its ultimate representative so that future calls to find will require less work is known as **path compression**. The benefit of path compression is further illustrated in the figures below.

Suppose we have a large disjoint set of people in our union-find container. Person A is the ultimate representative of everyone in the set. Person B has person A as their representative, person C has person B as their representative, and person D has person C as their representative.

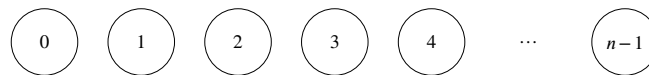


If we call `find_set()` on person D to identify the disjoint set they belong to, we would move up the tree from D and discover that person A is the ultimate representative. However, after this call to find, we would also discover that persons B, C, and D all share the same ultimate representative! If we ever call `find_set()` on B, C, or D, we would not want to repeat the work of walking up the tree to find the ultimate representative. With path compression, we can bring B, C, and D closer to their ultimate representative so that future calls to find run faster. This is done by updating each of their representatives to person A, as shown below:



13.3 Implementing a Union-Find Container

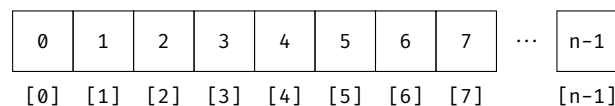
In this section, we will implement a union-find data structure using the concepts covered in the previous section. Suppose we are given a collection of n elements that we want to organize into disjoint sets:



As mentioned before, the union-find container will need to keep track of each element's representative, so we will initialize an underlying vector of indices as a private member variable of the `UnionFind` class. Each index of the vector stores the representative of the element at that index.

```
1 class UnionFind {
2     // vector that keeps track of representatives
3     // the value at index i is the index of the ith element's representative
4     std::vector<int32_t> reps;
5     ...
6 };
```

How should the `reps` vector be initialized? We do not want to default initialize all the representatives to 0, since that would imply that element 0 is the representative of every element in the collection. Instead, we want each element to be in its own disjoint set (of size 1) when the container is first initialized. This means that every element starts with *itself* as its representative (that is, the value at index i of the `reps` vector should be initialized to i).



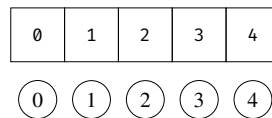
The following constructor takes in a size and initializes the representatives in the `reps` vector:

```
1 class UnionFind {
2     std::vector<int32_t> reps;
3 public:
4     // the reps vector should have each index initialized to the value of the index itself
5     UnionFind(int32_t size) {
6         reps.reserve(size);
7         for (int32_t i = 0; i < size; ++i) {
8             reps.push_back(i); // at the beginning, every node represents itself
9         } // for i
10    } // UnionFind()
11    ...
12 };
```

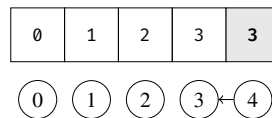
The `for` loop can also be condensed into one line using the `std::iota()` function:

```
std::iota(reps.begin(), reps.end(), 0);
```

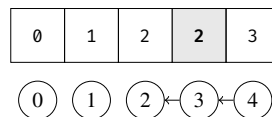

Now, we have to implement the `union_set()` and `find_set()` methods for our union-find container. To implement these, let's first look at how these operations should operate visually. Suppose we had five elements that all start off disjoint in their own set.



Suppose we called `union_set()` on elements 3 and 4. When this happens, 3 and 4 are merged into the same set. To reflect this change in our union-find container, we would need to change 4's representative to 3 to indicate that 3 and 4 are together in the same set (note that this choice to update 4 is arbitrary; we could have instead updated 3's representative to 4 if we wanted to).

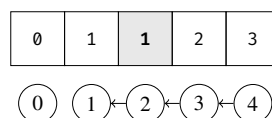


Now, suppose we union elements 2 and 3. To update our union-find container, we will update 3's representative to 2 to indicate that 2 and 3 belong to the same set.



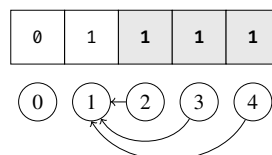
Notice that elements 2, 3, and 4 form a set in this container, with 2 as the ultimate representative of every element in this set. We know this because 2 is the only element in this set whose ultimate representative is itself.

Continuing on, if we unioned together elements 1 and 2, 2's representative would be updated to 1:

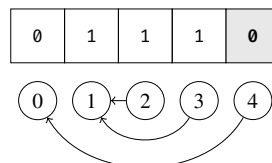


Now, let's call `find_set()` on element 4. Here, we want to return the ultimate representative of 4, which serves as an identifier for the disjoint set it belongs in. To do this, we will search up the chain of representatives from 4 until we encounter an element whose representative is itself. 4's representative is 3, 3's representative is 2, 2's representative is 1, and 1's representative is itself. Thus, 1 is the ultimate representative of 4.

However, that is not all we have to do. If we were to call `find` on 4 again, we would not want to look down its chain of representatives all over again. To address this, we can implement path compression and change the representative of every element along the path from 4 to 1 to the ultimate representative of 1.

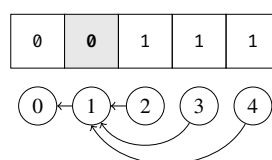


Now, let's union together 0 and 4. This is a bit tricky, since we cannot just change 4's representative to 0. Doing so would produce the following incorrect result:



If we just changed 4's representative, 4 would not longer be in the same set as 1, 2, and 3 after the union call. This is invalid, since 4 was moved out of its original set, which should not happen. Instead, the intended behavior of unioning 0 and 4 is for all five elements to be in the same set.

To fix this issue, we should update the representative of 4's *ultimate representative*, rather than 4 itself. Here, 1 is the ultimate representative of 4, so we modify 1's representative to 0. By doing so, 4 is able to join 0 without breaking its previous connections to 1, 2, and 3.



We now have enough information to implement the `find_set()` and `union_set()` functions of the union-find container. With `find_set()`, we would first find the ultimate representative of a given element by looping through its representatives until we find an element that represents itself. Then, we would utilize path compression to set every element along the chain of representatives we visited to this ultimate representative.

```

1 // find_set(x) returns the ultimate representative of x
2 int32_t find_set(int32_t x) {
3     int32_t current = x;
4     // pass 1: find the ultimate representative
5     while (reps[x] != x) {
6         x = reps[x];
7     } // while
8     // x now stores the ultimate representative
9     // pass 2: path compression, set all reps along path to x
10    while (reps[current] != x) {
11        int32_t next = reps[current];
12        reps[current] = x;
13        current = next;
14    } // while
15    // return the ultimate representative
16    return x;
17 } // find_set()

```

We can also use recursion to shorten the find process into the following lines of code:

```

1 int32_t find_set(int32_t x) {
2     if (x == reps[x]) {
3         return x;
4     } // if
5     reps[x] = find_set(reps[x]);
6     return reps[x];
7 } // find_set()

```

What about `union_set()`? To implement the union operation, we would want to find the ultimate representative of each element we want to union and update one representative to point to the other representative:

```

1 // union_set() unions the elements x and y into the same set
2 void union_set(int32_t x, int32_t y) {
3     int32_t x_rep = find_set(x);
4     int32_t y_rep = find_set(y);
5     reps[y_rep] = x_rep;
6 } // union_set()

```

In the above implementation, we arbitrarily decided that `y`'s representative will always be the one that is updated. This works, but it is not always ideal. In the next section, we will discuss methods to strategically select which representative gets updated in order to further improve the efficiency of union-find.

Example 13.1 The EECS 281 staff is testing out a brand new social media site to promote social interaction among EECS students! Suppose there are a total of n students that are invited to the site, and each student is assigned a unique integer ID from 0 to $n - 1$. You are given the number of students in the network, `num_students`, and a vector of activity logs, where each log stores an integer timestamp and the IDs of two students on the site:

```

1 struct Log {
2     int32_t timestamp;
3     int32_t id_A;
4     int32_t id_B;
5 };

```

Each `Log` object represents the time when students `id_A` and `id_B` became friends. The timestamp is represented in YYYYMMDD format (e.g., February 26, 2020 is represented as the integer 20200226). This format allows you to compare timestamps by directly comparing the integers themselves. Friendship is also symmetric: if A is friends with B, then B is friends with A.

Let's say that person A is *acquainted* with person B if A and B are in the same friend group; that is, if A is friends with B, or A is a friend of someone acquainted with B. Implement the `earliest_acquaintance()` function, which returns the earliest timestamp for which every person in the network is acquainted with every other person. Return -1 if there is no such earliest time.

```
int32_t earliest_acquaintance(std::vector<Log>& friendships, int32_t num_students);
```

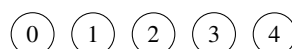
Example 1: Given `num_students = 5` and `friendships = [{20200229, 2, 3}, {20200227, 1, 4}, {20200303, 0, 3}, {20200228, 0, 4}, {20200301, 1, 2}, {20200226, 0, 2}]`, you would return 20200229, since that is the first timestamp for which all five students are acquainted (on the 29th, person 0 is direct friends with 2 and 4, acquainted to 1 via 4, and acquainted to 3 via 2).

Example 2: Given `num_students = 5` and `friendships = [{20200304, 1, 4}, {20200307, 0, 2}, {20200306, 3, 4}]`, you would return -1. This is because person 0 and person 2 are in a separate friend group and are never acquainted with 1, 3, or 4.

In this example, we want to find the earliest timestamp during which every student on the network is acquainted. The key insight to notice is that two students are acquainted if they are part of the same disjoint set! This makes the problem a good candidate for the union-find data structure.

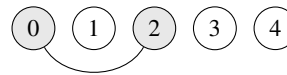
Let us look at the examples visually. Consider the first example: at the very beginning, no one is friends with anyone else, and thus none of the students start off as acquaintances.

Current timestamp: < 20200226



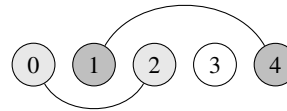
On the 26th, the earliest timestamp in the vector, 0 becomes friends with 2. As a result, these two students become part of the same friend group (i.e., the same disjoint set), and thus are acquainted.

Current timestamp: 20200226



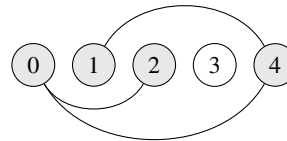
On the 27th, 1 becomes friends with 4, forming another distinct friend group:

Current timestamp: 20200227



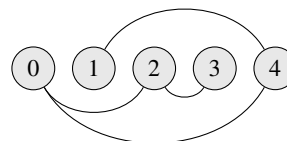
On the 28th, 0 becomes friends with 4. Notice that this combines the two existing friend groups; at this point, all but student 3 are acquainted.

Current timestamp: 20200228



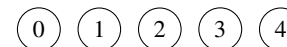
On the 29th, 2 becomes friends with 3, which ends up joining 3 with the friend group containing all the other students. There is only one disjoint set remaining after this friendship, so everyone is acquainted, and we can return the timestamp of 20220229.

Current timestamp: 20200229



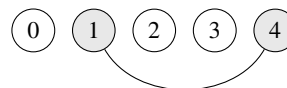
Let's consider the second example. Again, no one is friends with each other at the beginning, so each student starts off in their own disjoint set.

Current timestamp: < 20200304



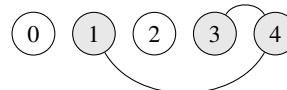
On the 4th, 1 becomes friends with 4, which puts them in the same disjoint set.

Current timestamp: 20200304



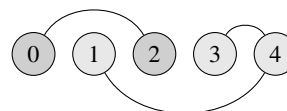
On the 6th, 3 becomes friends with 4, which adds 3 to the disjoint set containing 1 and 4.

Current timestamp: 20200306



On the 7th, 0 becomes friends with 2, forming their own disjoint set.

Current timestamp: 20200307



However, no additional friendships are made. There are multiple disjoint sets left over, so not every person in the network is acquainted with every other person (e.g., 0 and 2 are not acquainted with 1, 3, or 4). Because of this, we will return -1.

From these examples, we can come up with the steps needed to solve this problem. Our algorithm is as follows:

1. Sort the vector of logs by timestamp — this allows us to traverse the logs from the earliest timestamp to the latest timestamp.
2. Initialize the students using a union-find container, with each student starting off as its own representative (i.e., each student initially belongs in its own disjoint set).
3. Loop over the vector of logs. Every time you encounter a new friendship, union them together.
4. If only one disjoint set remains, return the timestamp at which this occurs. Otherwise, if all the logs are traversed and there are still multiple disjoint sets, return -1.

The final step is not as straightforward as it seems: how do we determine if there is only one disjoint set remaining? This can be done by identifying the number of ultimate representatives that exist in our union-find container. Recall that the ultimate representative is the element that serves as the "leader" of its disjoint set, and it has the unique property of being its own ultimate representative. However, if we want to count the number of elements whose ultimate representative is equal to itself, we would have to iterate over the entire union-find container — this would take $\Theta(n)$ time given n students. Is there a way to do better?

Instead of iterating over all the students and counting the number of ultimate representatives after every new friendship, a more efficient approach would be to count the number of times two students in *different* disjoint sets are unioned together. *This works because the total number of disjoint sets decreases by one every time two unacquainted students are unioned together.* Since we start off with n disjoint sets (as no one is friends with each other), we are guaranteed to have one disjoint set remaining after making exactly $n - 1$ connections between non-acquainted students. Notice that we can easily determine whether two students are unacquainted or not by calling `find_set()` on each of them and verifying that their ultimate representatives are different. By utilizing this strategy, we would only need to increment or decrement a counter after every friendship, which takes $\Theta(1)$ time.

There are two ways to keep track of the counter: you can either start the counter at 0 and increment the counter every time you union two unacquainted students, stopping once the counter reaches $n - 1$; or, you can start the counter at n and decrement the counter every time you union two unacquainted students, stopping the counter once you reach 1. The first approach counts the number of friendships that are made between unacquainted students, while the second approach counts the total number of disjoint sets that exist among the students. Both approaches would get you to the same answer. A full implementation of this solution is shown below:

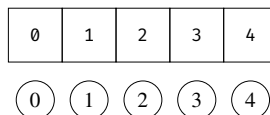
```

1  struct LogComp {
2      bool operator() (const Log& lhs, const Log& rhs) const {
3          return lhs.timestamp < rhs.timestamp;
4      } // operator()()
5  };
6
7  // find the representative (using path compression)
8  int32_t find(std::vector<int32_t>& reps, int32_t id) {
9      if (id == reps[id]) {
10         return id;
11     } // if
12     reps[id] = find(reps, reps[id]);
13     return reps[id];
14 } // find()
15
16 int32_t earliest_acquaintance(std::vector<Log>& friendships, int32_t num_students) {
17     std::sort(friendships.begin(), friendships.end(), LogComp()); // can also use a lambda
18     std::vector<int32_t> reps(num_students);
19     int32_t count = 0; // can also start at n and decrement
20     for (int32_t i = 0; i < num_students; ++i) {
21         reps[i] = i; // set each student to its own representative (can also use std::iota() to save lines)
22     } // for i
23     for (const Log& f : friendships) {
24         int32_t rep_A = find(reps, f.id_A);
25         int32_t rep_B = find(reps, f.id_B);
26         // if rep_A and rep_B are different, then A and B are part of different disjoint sets when they
27         // are unioned together, so increment the counter (or decrement if you started the counter at n)
28         if (rep_A != rep_B) {
29             reps[rep_B] = rep_A;
30             ++count;
31         } // if
32         // done when n - 1 connections are made (check count == 1 if decrementing from n)
33         if (count == num_students - 1) {
34             return f.timestamp;
35         } // if
36     } // for f
37     // return -1 if all logs are visited and fewer than n - 1 non-acquainted unions are made
38     return -1;
39 } // earliest_acquaintance()

```

13.4 Union-by-Size and Union-by-Rank (*)

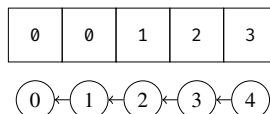
So far, our union implementation blindly sets the ultimate representative of the second disjoint set to that of the first. However, this process could still give a worst-case scenario where union and find both take $\Theta(n)$ time.



Suppose we called the following sequence of union operations:

- `union_set(3, 4)`
- `union_set(2, 3)`
- `union_set(1, 2)`
- `union_set(0, 1)`

If we did this, 4's representative would become 3, 3's representative would become 2, 2's representative would become 1, and 1's representative would become 0. This is shown accordingly:

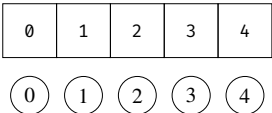


We have now run into a situation where the worst-case complexity of union and find both become $\Theta(n)$. If you wanted to call find on element 4, you would have to do a linear traversal of the entire vector to discover that 4's ultimate representative is 0.

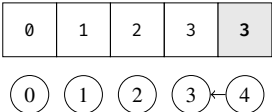
At this point, you may be wondering: shouldn't path compression handle this case? After we do the $\Theta(n)$ traversal, we would change the representatives of all the elements to 0 so that future calls to union and find become cheaper — as such, wouldn't we need to do the traversal only once? Although this is true, the issue is that we may have to do a $\Theta(n)$ traversal in the first place! Even though path compression improves the *amortized* cost of union and find, it does not improve the *worst-case* cost of either operation. That is, path compression does not prevent you from ending up with a worst-case "stick" formation of elements, such as in the example above. To address this, there are two optimization strategies we can use: *union-by-size* and *union-by-rank*.

※ 13.4.1 Union-by-Size (※)

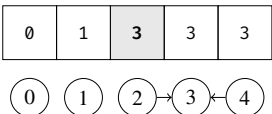
In a **union-by-size** approach, we additionally keep track of the sizes of each disjoint set in our union-find data structure. When two disjoint sets are unioned together, we always update the representative of the smaller set (ties are broken arbitrarily). Consider the same sequence of union operations from above, but this time using union-by-size to determine which representative is updated.



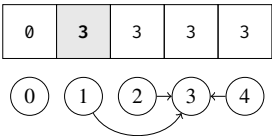
When we union 3 and 4, we are able to arbitrarily select which representative we want to update (since both sets are the same size). In this case, let's suppose we updated 4's representative to be 3.



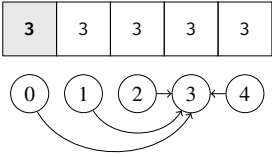
Now, we want to union 2 and 3. The disjoint set that 2 belongs to has a size of 1, and the disjoint set that 3 belongs to has a size of 2. Since 2's disjoint set is smaller, we assign 2's representative to 3's ultimate representative, or 3.



Now, we want to union 1 and 2. The disjoint set that 1 belongs to has a size of 1, and the disjoint set that 2 belongs to has a size of 3. Since 1's disjoint set is smaller, we assign 1's representative to 2's ultimate representative, or 3.



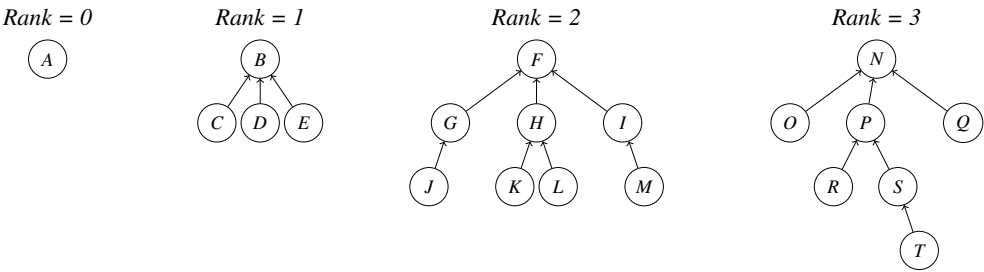
Now, we want to union 0 and 1. The disjoint set that 0 belongs to has a size of 1, and the disjoint set that 2 belongs to has a size of 4. Since 0's disjoint set is smaller, we assign 0's representative to 1's ultimate representative, or 3.



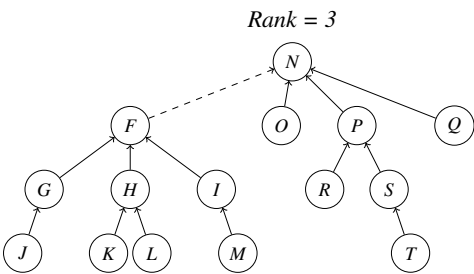
Notice here that we no longer have a stick-like chain of representatives. In fact, if union-by-size is used, the worst-case time complexity of a single union or find call drops from $\Theta(n)$ to $\Theta(\log(n))$, where n is the number of elements in the container. This is because union-by-size ensures that the longest possible chain of representatives has a length of at most $\Theta(\log(n))$.

※ 13.4.2 Union-by-Rank (※)

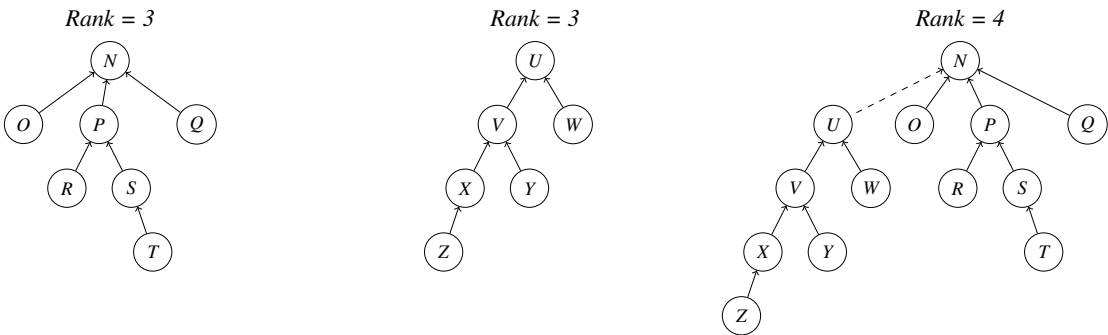
Another optimization technique is **union-by-rank**, which keeps track of the approximate height (or *rank*) of each disjoint set in the data structure instead of its size. When two disjoint sets are unioned together under the union-by-rank optimization approach, we always update the representative of the set with the smaller rank. Some examples of rank are shown below.



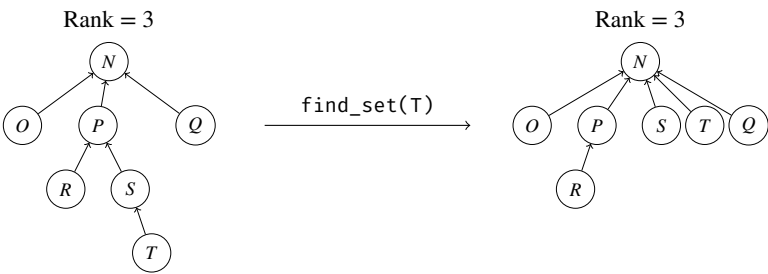
For instance, if we wanted to union elements F and N , we would update F 's representative to N because F 's disjoint set has a smaller rank. The rank of N , however, would still remain 3 after the sets are joined together.



Rank is incremented if two disjoint sets with the same rank are unioned together. When unioning two disjoint sets with the same rank, it does not matter which representative is updated to the other.



Note that we consider rank as an *approximation* for the longest representative chain that exists between an element and its ultimate representative. This is because rank is *not* recomputed if path compression changes the actual maximum length. For example, if we call `find_set(T)` on the following disjoint set using union-by-rank, both S and T 's representatives would change to N , but the rank of the entire set would still be 3.



※ 13.4.3 Union-Find Complexity Analysis ※

So far, we have discussed several union-find optimization strategies, including path compression, union-by-size, and union-by-rank. Path compression can be used to move elements closer to their ultimate representatives, which speeds up future calls to union and find. Union-by-size or union-by-rank can be used to build the connections in a way that prevents the length of any representative chain from growing beyond $\Theta(\log(n))$. However, how do these optimizations impact the overall time complexities of union and find?

The answer is not trivial, so we will not be going into the full details here. If we look at a union-find container that only implements path compression but not union-by-rank or union-by-size, then the time complexity of $n - 1$ union operations and m find operations is bounded by $O(n + m \log(n))$ for $m < n$, and $O(m \log(n))$ for $m \geq n$. On a similar vein, if we have a union-find container that only implements union-by-rank or union-by-size, but not path compression, then the time complexity of $n - 1$ union operations and m find operations is also bounded by a worst-case time complexity of $O(m \log(n))$. This is because the costs of find and union are both bounded by the largest rank or size among the disjoint sets, which is itself bounded by $O(\log(n))$.

However, if we combine path compression with either union-by-size or union-by-rank, the time complexity of any m union-find operations drops down to $\Theta(m\alpha(n))$, which also means that the amortized costs of find and union both become $\Theta(\alpha(n))$. Here, $\alpha(n)$ represents the **inverse Ackermann function**, which is a function that grows so slowly that its value can practically be treated as a constant for reasonable values of n . In other words, if path compression is combined with union-by-size or union-by-rank, `find_set()` and `union_set()` can both be done in (essentially) amortized constant time!²

Disjoint sets and union-find have practical use cases in several important computer science problems, such as image processing or counting connected components. We will revisit the concept of union-find in chapter 20 when we discuss Kruskal's algorithm, which can be used to find the minimum cost required to connect all nodes in a graph.

²The analyses in this section were obtained from the research paper "Worst-Case Analysis of Set Union Algorithms" by Tarjan and van Leeuwen (1984), and chapter 21 of "Introduction to Algorithms (Third ed.)" by Cormen, Leiserson, Rivest, and Stein (2009).

Practice Quiz

Disclaimer: These practice questions are not official and may not have been vetted for course material. You may use these for practice, but you should prioritize official resources from current staff for a more accurate depiction of what you need to know for assignments and exams.

1. Let F represent the set of all students enrolled in the Fall 2017 semester of EECS 281, let W represent the set of all students enrolled in the Winter 2018 semester of EECS 281, and let S represent the set of all students enrolled in the Spring 2018 semester of EECS 281. Which of the following students would **NOT** be included in the set $(F \cap W) \cup S$?
 - A) A student who took EECS 281 only once and passed during the Winter 2018 semester
 - B) A student who took EECS 281 only once and passed during the Spring 2018 semester
 - C) A student who took but did not pass EECS 281 during the Fall 2017 semester and retook it during the Winter 2018 semester
 - D) A student who took but did not pass EECS 281 during the Winter 2018 semester and retook it during the Spring 2018 semester
 - E) More than one of the above
2. You are given two **sorted** vectors of integers, both of size n , and you want to devise an algorithm that can identify all items in one vector but not the other. Assuming that you are not allowed to create any additional containers, what is the worst-case time complexity of the most efficient algorithm that is attainable for solving this problem?
 - A) $\Theta(1)$
 - B) $\Theta(\log(n))$
 - C) $\Theta(n)$
 - D) $\Theta(n \log(n))$
 - E) $\Theta(n^2)$
3. You are given two **unsorted** vectors of integers, both of size n , and you want to devise an algorithm that can identify all items are either the first or second vector, but not both. Assuming that you are not allowed to create any additional containers, what is the worst-case time complexity of the most efficient algorithm that is attainable for solving this problem?
 - A) $\Theta(1)$
 - B) $\Theta(\log(n))$
 - C) $\Theta(n)$
 - D) $\Theta(n \log(n))$
 - E) $\Theta(n^2)$
4. The STL's set algorithms of `std::set_union()`, `std::set_intersection()`, etc., rely on which of the following assumptions about their input to produce the desired behavior?
 - I. The two input ranges passed in must already have their values sorted.
 - II. The two input ranges passed in must be stored in containers that support random access.
 - III. The two input ranges passed in must contain the same number of elements.
 - A) I only
 - B) III only
 - C) I and II only
 - D) I and III only
 - E) I, II, and III
5. Which of the following could be a practical application of the union-find data structure?
 - A) Implementing a image editing program that identifies and updates regions of an image with a similar color
 - B) Implementing a networking program that identifies groups of servers that can communicate with each other
 - C) Implementing an algorithm that can be used to identify people in a social network with similar interests
 - D) Implementing an algorithm that counts the number of connected components in a graph
 - E) More than one of the above
6. Which of the following statements is **TRUE** about an implementation of the union-find container that forces every element in the data structure to keep track of the *ultimate* representative of its disjoint set, if no path compression is used?
 - A) The find operation may take up to $\Theta(n)$ time, given n elements in the union-find container
 - B) The union operation may take up to $\Theta(n)$ time, given n elements in the union-find container
 - C) The amortized time complexities of find and union are both $\Theta(n)$, given n elements in the union-find container
 - D) The amortized time complexities of find and union are both $\Theta(\alpha(n))$, given n elements in the union-find container, where $\alpha(n)$ denotes the inverse Ackermann function
 - E) More than one of the above
7. Consider a union-find container with n elements, each with its representative set to itself. Assuming path compression is used, and only one call to `union_set()` is invoked with two arbitrary elements in this container (with no other union or find operations completed before or after), what is the largest possible number of elements that could have their representatives modified after this function call?
 - A) 0
 - B) 1
 - C) 2
 - D) $n - 1$
 - E) n

8. Consider a union-find container where, in addition to remembering one of its representatives, each element also keeps track of the *size* of the disjoint set that it belongs to. When elements from two disjoint sets are unioned together, the representative of the smaller set is always updated to the representative of the larger set (with ties broken arbitrarily). Under this implementation, what is the worst-case time complexity of a single call to `find_set()` on this union-find container? Note that $\alpha(n)$ represents the inverse Ackermann function.
- A) $\Theta(1)$
 - B) $\Theta(\alpha(n))$
 - C) $\Theta(\log(n))$
 - D) $\Theta(n)$
 - E) $\Theta(n\alpha(n))$

9. The following represents the state of a union-find container:

Item	0	1	2	3	4	5	6	7	8	9
Representative	5	6	7	2	1	3	6	9	8	8

Among these 10 elements, how many disjoint sets are there?

- A) 0
 - B) 1
 - C) 2
 - D) 3
 - E) 4
10. Consider the same union-find container as in question 9. Suppose you wanted to merge all of the disjoint sets among these elements into a single set. What is the *minimum* number of union operations necessary to accomplish this?
- A) 0
 - B) 1
 - C) 2
 - D) 3
 - E) 4
11. Consider the same union-find container as in question 9. Suppose you used the path compression approach when you implemented union-find. If you were to call the find function on item 0, which of the following statements would be **TRUE**?
- A) 0's representative would become 3
 - B) 5's representative would become 7
 - C) 4's representative would become 6
 - D) 2's representative would become 8
 - E) More than one of the above
12. Consider the same union-find container as in question 9. Suppose you used the path compression approach when you implemented union-find. Which of the following operations would **NOT** change any of the representatives given in the above table?
- A) Calling find on item 7
 - B) Calling find on item 4
 - C) Calling find on item 3
 - D) Calling find on item 1
 - E) None of the above
13. You are given a union-find data structure, which implements path compression. There are 281 elements in the container, and 203 of these elements have themselves as their ultimate representative. What is the largest possible size that a disjoint set within this container can have?
- A) 77
 - B) 78
 - C) 79
 - D) 202
 - E) 203
14. You are given a union-find data structure, which implements path compression (but no other optimizations). There are 370 elements in the container, and 280 of these elements have themselves as their ultimate representative. You perform one call to `find_set()` on this container with an arbitrary element (with no other operations completed). What is the largest possible number of elements that could potentially have its representative changed with this call to `find_set()`?
- A) 90
 - B) 91
 - C) 279
 - D) 280
 - E) 370

15. You are given a union-find data structure, which implements path compression. There are five elements in this container: A, B, C, D, and E. Item A has a representative of E, item B has a representative of A, item C has a representative of B, and item D has a representative of D. After calling `union_set()` on items C and D, you notice that the ultimate representative of item D was changed from D to E. Which of the following statements is **TRUE**?
- A) Before the call to `union_set()`, there were three disjoint sets in this union-find container
 - B) Before the call to `union_set()`, items D and E were part of the same disjoint set
 - C) Before the call to `union_set()`, items C and E were part of different disjoint sets
 - D) Before the call to `union_set()`, the ultimate representative of item E was C
 - E) Before the call to `union_set()`, the ultimate representative of item E was E

16. You are given an array of strings `equations` that represent relationships between variables, where `equations[i]` is a string of length 4 that takes on one of two different forms: "`x=y`" or "`x!=y`" (where `x` and `y` can be interchanged with any other character from `a` to `z`, which you may assume will always be lowercase). Implement a function that returns whether it is possible to satisfy all the given equations.

For example, if you are given the equations "`a==b`", "`b==c`" and "`a!=c`", you would return `false`, since there is no way to satisfy all three of these equations. However, if you were just given the first two equations, you would return `true`.

```
bool are_equations_satisfiable(const std::vector<std::string>& equations);
```

17. You are given a list of n computers, labeled from 0 to $n - 1$. You are also given a two-dimensional vector of integers `firewall`, where `firewall[i] = [x, y]` indicates that computers `x` and `y` cannot be connected at all, either directly or indirectly through other computers within the same network.

Initially, none of the computers are connected to each other. You are given a list of connections that you are requested to make in a vector `requests`, where `requests[i] = [u, v]` is a request to add a network connection between computers `u` and `v`. A connection request is successful if `u` and `v` can be connected without breaking any of the restrictions specified in the `firewall` vector. Each request is processed in the order specified, and upon a successful request, the two computers involved retain their direct connection for all future requests.

Return a boolean array `result`, where `result[i]` is true if the i^{th} request is successful, and false if it is not. You may assume that requests to connect two computers that are directly connected are trivially successful.

For example, given $n = 3$ computers, the following firewall restrictions: `[[0, 1]]`, and the following requests: `[[1, 2], [0, 2]]`, you would return `[true, false]`. This is because the first request is successful, since connecting computers 1 and 2 does not break any firewall rules. However, the second request is not successful, since computer 2 is now connected to computer 1 via the first request, and connecting 2 with 0 would break the restriction that computers 0 and 1 cannot be connected in any fashion, both directly and indirectly.

```
std::vector<bool> connection_requests(int32_t n, const std::vector<std::vector<int32_t>>& firewall,
                                     const std::vector<std::vector<int32_t>>& requests);
```

18. You are given a list of n computers, labeled from 0 to $n - 1$. There are two ways you can link these computers with each other:
- *Physical link*: a link that connects two computers directly.
 - *Wireless link*: allows a computer to communicate directly with *any* other computer that also has a wireless link. For example, if computers 0, 1, and 2 are given wireless links, then they would all be able to communicate with each other.

Two computers do not need to be directly connected to be able to receive messages from each other. For example, if computer 0 has a physical link with computer 1, and computer 1 has a physical link with computer 2, then you would still be able to send a message from computer 0 to 2. Similarly, if computer 0 has a physical link with computer 1, and both computers 1 and 2 support wireless links, then computer 0 would also be able to send a message to computer 2 using a direct connection to 1 and a wireless connection to 2.

Implement the following function, which takes in the number of computers n , two computers i and j (which are indices from 0 to $n - 1$), and a 2-D vector of links, where `links[d]` includes all the links that will be installed on day d , and returns the day number on which computer i will be able to communicate with computer j (or `-1` if not possible). Each `Link` entity is defined as follows, for two computers `src` and `dest`. If `dest` is equal to `-1`, then `src` will be given a wireless link; otherwise a physical link between `src` and `dest` will be added. You may also use the pre-implemented `UnionFind` data structure shown below in your solution.

```
// Pre-implemented data structure that you can use in your solution (no need to implement this)
class UnionFind {
public:
    UnionFind(int32_t n); // creates a union-find container with n elements
    // Returns whether i and j are in the same disjoint set
    bool connected(int32_t i, int32_t j);
    // Merges i and j into the same disjoint set if they are not in one already
    void union_set(int32_t i, int32_t j);
};

// Link entity that indicates a request to link src with dest
// If dest is -1, a wireless link is added to src instead
struct Link { int32_t src; int32_t dest; };

// Implement this function: returns earliest day computers i and j can communicate with each other
int32_t earliest_connection_day(int32_t n, int32_t i, int32_t j, const std::vector<std::vector<Link>>& links);
```

» « Quiz Solutions » «

1. **The correct answer is (A).** A student who took EECS 281 once and passed in the spring semester would be included in S , a student who took EECS 281 in both the fall and winter semesters would be included in $F \cap W$, and a student who took EECS 281 in both winter and spring would be included in S . Only a student who took EECS 281 once in the winter would be included in neither $F \cap W$ nor S .
2. **The correct answer is (C).** Since the vectors are sorted, you can find the set difference by simply performing a linear pass of the two vectors, using the sorted order to determine if an element exists in one vector but not the other. See the implementation of `set_difference()` in section 13.1.2 for an example of how this can be done.
3. **The correct answer is (D).** If you are not allowed to create any additional containers, the best time complexity you can do is $\Theta(n \log(n))$, since you would need to presort the vectors before you can solve the problem using a linear pass (similar to the solution for question 2). Note that the best solution if both vectors remain unsorted takes $\Theta(n^2)$ time, since for each value in one vector, you would need to perform a linear pass over the other vector to identify if it also exists there. This is inferior to the $\Theta(n \log(n))$ cost of sorting the vector beforehand.
4. **The correct answer is (A).** Only statement I is required for the STL's set algorithms, as they rely on the assumption that the provided values are in sorted order. Statement II is false since these methods take in input iterators, and statement III is false because the two provided iterator ranges can be of different sizes (what happens to any extra values in the larger range depends on the method called).
5. **The correct answer is (E).** The union find data structure is most useful for working with disjoint sets. All of the options are examples where this may be applicable, since they all deal with identifying or merging elements into disjoint sets.
6. **The correct answer is (B).** If every element in the data structure is forced to keep track of its ultimate representative, a single union call may require $\Theta(n)$ elements to have their representatives updated. This is the "quick-find" implementation of union find, which allows `find_set()` to be done in constant time at the expense of a more expensive `union_set()` call.
7. **The correct answer is (B).** An element has a representative equal to itself if it is the ultimate representative of its disjoint set. Since every element has its representative equal to itself, each element must be in its own disjoint set — because of this, a call to `union_set()` would merge two sets of size one, and thus will only be able to update one representative.
8. **The correct answer is (C).** The worst-case performance of `find_set()` depends on length of the representative chain from an element to its ultimate representative. Normally, this would be $\Theta(n)$ since you could have a representative chain in the form of a stick (where A is an ultimate representative, B's representative is A, C's representative is B, etc.). However, if you enforce that the smaller set is always updated to be part of the larger set after a union, then the maximum length of a representative chain drops down from $\Theta(n)$ to $\Theta(\log(n))$, which also reduces the worst-case time complexity of a single `find_set()` call.
9. **The correct answer is (C).** There are two disjoint sets: $\{0, 2, 3, 5, 7, 8, 9\}$ and $\{1, 4, 6\}$. One strategy is to count the number of elements with a representative equal to itself, since this is the number of ultimate representatives that exist in the container.
10. **The correct answer is (B).** Since there are two disjoint sets, only one union operation is needed to bring the total number of disjoint sets down to one (by calling union on two elements in different sets). In general, if there are n disjoint sets, at least $n - 1$ calls to union are needed to merge everything into a single set.
11. **The correct answer is (D).** Using the path-compression approach, if we were to call `find_set()` on an element j , we would traverse all elements on the way to its ultimate representative k and change all these elements to have a representative of k . Since we called `find_set()` on element 0, we follow the chain all the way to 0's ultimate representative of 8:

$$0 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 7 \rightarrow 9 \rightarrow 8.$$
 Here, all of these elements would have their representatives changed to 8.
12. **The correct answer is (D).** Calling `find_set()` on an element would not change its representative if there are no intermediaries between such an element and its ultimate representative. This is only true for element 1, as 1's representative is 6, which is its own representative. Calling find on element 7 would change its representative to 8, calling find on element 4 would change its representative to 6, and calling find on element 3 would change its representative to 8 (as well as others down the chain to 8).
13. **The correct answer is (C).** If there are 281 elements in the container, and 203 have themselves as their ultimate representative, then there must be 203 disjoint sets. If 202 of these sets have only one element, then there are $281 - 202 = 79$ elements remaining to form the last disjoint set. The size cannot be larger than 79, or it would be impossible for there to be 203 disjoint sets in total.
14. **The correct answer is (A).** Since path compression is supported, the largest possible number of elements that could have their representatives updated is equal to the size of the largest disjoint set, minus 1 (since the ultimate representative of the set does not get updated), which could happen if you have a representative chain in the form of a stick, and you call `find_set()` on the ultimate representative and farthest element along the stick. The largest disjoint set can have a total of $370 - 279 = 91$ elements, so the greatest number of representatives that could potentially be updated is 90.
15. **The correct answer is (E).** Option (A) is false because, from the existing representatives, there were only two disjoint sets before the union: $\{A, B, C, E\}$ and $\{D\}$. This also means (B) and (C) are false as well. Of the remaining options, (E) is correct because the disjoint set containing $\{A, B, C, E\}$ would not have an ultimate representative otherwise, as none of A, B, or C have its representative equal to itself.

16. This problem can be solved using a union-find container. To do so, we would first iterate over all the equality (==) equations and merge them into the same disjoint set. Then, we iterate over all the non-equality (!=) equations and call find to determine if they belong to different disjoint set. If not, then we have a contradiction, so we return false. Otherwise, if all the != equations involve variables that belong in different disjoint sets, we return true. One implementation of this solution is shown below:

```

1  int32_t find(std::vector<int32_t>& reps, int32_t id) {
2      if (id == reps[id]) {
3          return id;
4      } // if
5      reps[id] = find(reps, reps[id]);
6      return reps[id];
7  } // find()
8
9  bool are_equations_satisfiable(const std::vector<std::string>& equations) {
10     std::vector<int32_t> reps(26); // 'a' to 'z'
11     for (int32_t i = 0; i < 26; ++i) {
12         reps[i] = i;
13     } // for i
14     for (const std::string& equation : equations) {
15         char var1 = equation[0] - 'a';
16         char var2 = equation[3] - 'a';
17         if (equation[1] == '=') {
18             reps[find(reps, var1)] = find(reps, var2);
19         } // if
20     } // for equation
21     for (const std::string& equation : equations) {
22         char var1 = equation[0] - 'a';
23         char var2 = equation[3] - 'a';
24         if (equation[1] == '!' && find(reps, var1) == find(reps, var2)) {
25             return false;
26         } // if
27     } // for equation
28     return true;
29 } // are_equations_satisfiable()

```

17. This problem can also be solved using a union-find container, since we need to be able to quickly identify if two computers are on the same network (i.e., disjoint set). The solution is pretty similar to the previous problem: we iterate over the requests and check if we can make a connection without breaking any restrictions (using a call to find). If we can, then a connection is made (using a call to union). One implementation of this solution is shown below:

```

1  int32_t find_set(std::vector<int32_t>& reps, int32_t id) {
2      if (id == reps[id]) return id;
3      reps[id] = find_set(reps, reps[id]);
4      return reps[id];
5  } // find_set()
6
7  void union_set(std::vector<int32_t>& reps, int32_t x, int32_t y) {
8      int32_t x_rep = find_set(reps, x);
9      int32_t y_rep = find_set(reps, y);
10     reps[y_rep] = x_rep;
11 } // union_set()
12
13 std::vector<bool> connection_requests(int32_t n, const std::vector<std::vector<int32_t>>& firewall,
14                                     const std::vector<std::vector<int32_t>>& requests) {
15     std::vector<int32_t> reps(n);
16     for (int32_t i = 0; i < n; ++i) {
17         reps[i] = i;
18     } // for i
19     std::vector<bool> result;
20     for (const std::vector<int32_t>& request : requests) {
21         bool can_connect = true;
22         int32_t comp1 = find_set(reps, request[0]);
23         int32_t comp2 = find_set(reps, request[1]);
24         for (const std::vector<int32_t>& f : firewall) {
25             int32_t fw_rep1 = find_set(reps, f[0]);
26             int32_t fw_rep2 = find_set(reps, f[1]);
27             if ((comp1 == fw_rep1 && comp2 == fw_rep2) || (comp2 == fw_rep1 && comp1 == fw_rep2)) {
28                 can_connect = false;
29                 break;
30             } // if
31         } // for f
32         result.push_back(can_connect);
33         if (can_connect) {
34             union_set(reps, comp1, comp2);
35         } // if
36     } // for request
37     return result;
38 } // connection_requests()

```

18. One possible solution is shown below. We will use the union-find container to initialize a container of $n + 1$ elements, where indices 0 to $n - 1$ represent the n computers, and index n represents a wireless link (e.g., any computer in the same disjoint set as index n is part of the wireless network, and thus can communicate with each other). For each day, we iterate over the links and union indices src and $dest$ if a physical link is specified, or index src and index n if a wireless link is specified. Then, if the input computers i and j are connected at the end of the day, we return that day number from our function.

```

1  int32_t earliest_connection_day(int32_t n, int32_t i, int32_t j, const std::vector<std::vector<Link>>& links) {
2      UnionFind network(n + 1);
3      for (int32_t day_num = 0; day_num < links.size(); ++day_num) {
4          for (const Link& link : links[day_num]) {
5              if (link.dest == -1) {
6                  network.union_set(link.src, n);
7              } // if
8              else {
9                  network.union_set(link.src, link.dest);
10             } // else
11         } // for link
12         if (network.connected(i, j)) {
13             return day_num;
14         } // if
15     } // for day_num
16     return -1;
17 } // earliest_connection_day()

```